

V0720 LABORATORIO ABIERTO: DISEÑO
SPRINT 2 - DOCUMENTO TÉCNICO/CONCEPTUAL



Nombre del Proyecto: FamiGest
Integrantes del equipo: Morales Rios Maria Fernanda



LABORATORIO ABIERTO: DISEÑO

Centro Universitario de Ciencias Exactas e Ingenierías

SPRINT 2 –DISEÑO TÉCNICO/CONCEPTUAL

Duración: Semanas 6–9

1. Arquitectura de la solución

La solución se implementa como una app móvil que integrará dos flujos principales:

- Flujo de aprendizaje (microlecciones, ejercicios y progreso)
- Flujo de evaluación automática (captura por cámara → extracción de landmarks → clasificación → retroalimentación).

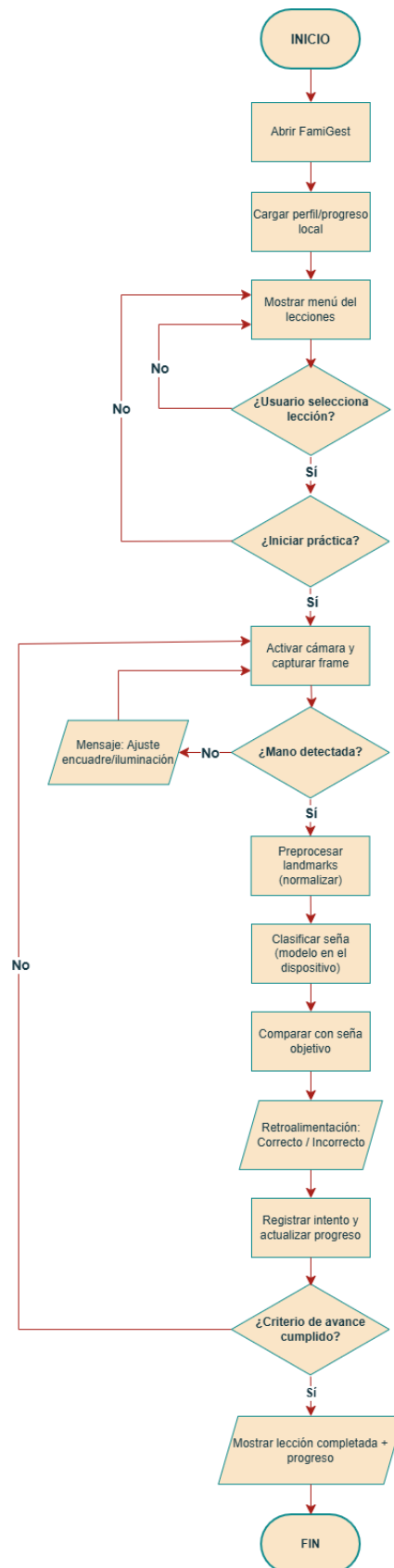
Todo el procesamiento crítico de reconocimiento se ejecuta en el dispositivo para reducir dependencias de conectividad y mitigar riesgos de privacidad.

Diagrama de flujo



LABORATORIO ABIERTO: DISEÑO

Centro Universitario de Ciencias Exactas e Ingenierías





2. Componentes principales

A continuación, se describen los componentes del sistema, su función técnica, relación con otros componentes y qué parte del proceso resuelven.

2.1 Interfaz de usuario (UI) y navegación

- Función: Presentar lecciones, guiar la práctica y mostrar retroalimentación y progreso.
- Relación: Se comunica con el motor de lecciones (para cargar contenido y ejercicios) y con el motor de retroalimentación (para mostrar resultado).
- Qué resuelve: Reduce fricción de uso y facilita el aprendizaje guiado (selección, claridad de avance, repetición).

2.2 Motor de lecciones (gestión de contenido y ejercicios)

- Función: Administrar las 3 lecciones del Producto Mínimo Viable (MVP), sus objetivos, ejercicios y criterios de avance.
- Relación: Alimenta a la UI con el contenido; invoca el módulo de práctica cuando corresponde; recibe resultados del registro de progreso.
- Qué resuelve: Estructura pedagógica mínima (aprender → practicar → avanzar).

2.3 Módulo de práctica con cámara (captura)

- Función: Activar cámara, capturar frames y entregarlos al módulo de extracción.
- Relación: Recibe “iniciar práctica” desde UI; entrega frames a extracción de landmarks.
- Qué resuelve: Entrada principal del sistema para evaluar ejecución real del gesto.

2.4 Extracción de landmarks de mano

- Función: Detectar la mano y obtener puntos clave (landmarks) que describen posición/orientación de dedos.
- Relación: Recibe frames del módulo de cámara; entrega landmarks al preprocesamiento.
- Qué resuelve: Reduce la entrada visual a una representación compacta y más manejable para clasificación.

2.5 Preprocesamiento de landmarks

- Función: Normalizar la representación (por ejemplo, centrar en un punto base, escalar, filtrar ruido).
- Relación: Consume landmarks; produce un vector de características para el clasificador.
- Qué resuelve: Mejora estabilidad ante variaciones de distancia/posición de la mano.

2.6 Clasificador de seña (modelo ligero on-device)



LABORATORIO ABIERTO: DISEÑO

Centro Universitario de Ciencias Exactas e Ingenierías

- Función: Estimar qué seña está realizando el usuario a partir de las características.
- Relación: Recibe vector preprocesado; envía predicción al motor de retroalimentación.
- Qué resuelve: Permite evaluación automática en tiempo real sin depender de terceros.

2.7 Motor de retroalimentación

- Función: Convertir la predicción en una respuesta entendible: correcto/incorrecto y sugerencia de reintento.
- Relación: Recibe predicción del clasificador y el objetivo de la actividad del motor de lecciones; envía resultado a UI y registro.
- Qué resuelve: Ataca el núcleo del problema (certeza al practicar).

2.8 Registro de progreso (almacenamiento local)

- Función: Guardar por usuario: intentos, aciertos, lecciones completadas y fecha/hora básica.
- Relación: Recibe eventos del motor de retroalimentación y del motor de lecciones; alimenta UI (progreso).
- Qué resuelve: Permite seguimiento del aprendizaje sin infraestructura externa.

3. Requerimientos técnicos básicos

Los requerimientos se derivan de la necesidad de capturar video en tiempo real, extraer landmarks y ejecutar clasificación local con retroalimentación inmediata.

3.1 Capacidades mínimas del dispositivo

- Cámara funcional con captura de video estable (suficiente para detectar mano y landmarks en tiempo real).
- Capacidad de procesamiento para ejecutar extracción de landmarks y clasificación local sin latencia excesiva (flujo interactivo).
- Memoria disponible para ejecutar la app y cargar el modelo ligero y recursos de lecciones (contenido del MVP).

3.2 Condiciones de operación

- Iluminación suficiente y relativamente uniforme para que la mano se detecte con estabilidad.
- Mano visible y completa dentro del encuadre durante la práctica (sin oclusiones relevantes).



LABORATORIO ABIERTO: DISEÑO

Centro Universitario de Ciencias Exactas e Ingenierías

- Fondo no necesariamente “limpio”, pero sin condiciones extremas que impidan distinguir la mano (por ejemplo, oscuridad total).

3.3 Restricciones del proceso (para el MVP)

- Alcance limitado a señas estáticas en 3 lecciones, para mantener consistencia del reconocimiento.
- Retroalimentación binaria (correcto/incorrecto) como salida mínima, priorizando claridad sobre complejidad.

3.4 Información necesaria para operar

- Definición de cada ejercicio: seña objetivo esperada y criterio de avance.
- Recurso visual por lección (video/imagen/animación de referencia).
- Modelo/plantilla de referencia (archivo del clasificador o reglas) cargado localmente.

4. Decisiones técnicas clave

4.1 Reconocimiento on-device vs reconocimiento por servidor

Se eligió reconocimiento en el dispositivo para evitar dependencia de internet, reducir latencia y disminuir riesgos de privacidad (no es necesario transmitir imágenes o video). La alternativa por servidor se descartó por costo operativo, latencia y complejidad de mantenimiento en un piloto individual.

4.2 Landmarks vs clasificación directa sobre imagen

Se eligió usar landmarks de mano como representación intermedia porque reduce dimensionalidad y hace la clasificación más manejable, especialmente en un MVP. Clasificar directamente con imagen suele requerir más datos, más cómputo y puede ser más sensible a fondos/iluminación.

4.3 Alcance inicial a señas estáticas (3 lecciones)

Se acotó a señas estáticas porque las señas dinámicas agregan complejidad (secuencia temporal, variaciones de movimiento) y elevan el riesgo de inestabilidad en un primer piloto. Esta decisión prioriza consistencia y comprensibilidad de la retroalimentación.

4.4 Retroalimentación binaria (correcto/incorrecto) en el MVP

Se prioriza una salida simple y clara para el usuario. Una calificación continua (porcentaje, “casi”) puede confundir o requerir calibración adicional. El compromiso aceptado es sacrificar granularidad a cambio de claridad y estabilidad.

4.5 Almacenamiento local de progreso en fase piloto



LABORATORIO ABIERTO: DISEÑO

Centro Universitario de Ciencias Exactas e Ingenierías

Se eligió almacenar progreso localmente para reducir infraestructura y riesgos. La sincronización con backend se considera ampliación futura cuando el alcance y usuarios aumenten.

5. Justificación técnica inicial

La solución es técnicamente coherente bajo principios básicos de ingeniería de software y visión por computadora aplicada:

- Separación por módulos: UI/motor de lecciones/visión/clasificación/retroalimentación/almacenamiento. Esto reduce acoplamiento, facilita pruebas y permite evolucionar el sistema por partes.
- Pipeline de percepción y decisión: captura → extracción de características (landmarks) → preprocesamiento → clasificación → retroalimentación. Este patrón es estándar en sistemas de reconocimiento por computadora.
- Inferencia local: ejecutar el reconocimiento en el dispositivo reduce latencia, dependencia de red y exposición de datos, mejorando la confiabilidad operativa en contextos domésticos.
- Diseño centrado en retroalimentación: la arquitectura prioriza el ciclo de práctica con feedback inmediato, que es el componente técnico que más incide en reducir incertidumbre del usuario.

6. Conclusión

Definir un MVP permite concentrar el desarrollo en lo esencial: una versión mínima pero funcional que demuestre el flujo completo del sistema y valide si la solución realmente ataca el problema central. En FamiGest, acotar el alcance (por ejemplo, 3 lecciones y prueba con 2 usuarios) reduce riesgos, facilita completar el prototipo y genera evidencia temprana para decidir qué mejorar antes de escalar a más contenido o a funciones más complejas.